

WEB APPLICATION SECURITY ASSESSMENT AND SQL INJECTION TESTING

Module 5 Assignment — DVWA

Prepared by: Chandrapal Singh Panwar

Date: 02 October 2026

Target: Local DVWA — <http://127.0.0.1:42001/>

1. Executive Summary

This assessment was conducted against a locally hosted Damn Vulnerable Web Application (DVWA) in a controlled laboratory environment. The objective was to assess SQL injection, automated SQLMap exploitation, database enumeration, data exposure, parameter tampering, blind SQL injection, and authentication-bypass behavior.

Testing confirmed SQL injection in the GET parameter 'id'. Manual testing returned multiple records. SQLMap identified MySQL/MariaDB and confirmed UNION-based and time-based blind injection techniques. The dvwa database and its tables were enumerated, and the users table was successfully extracted. Boolean-based blind SQL injection was also demonstrated.

2. Scope and Environment

Item	Details
Application	Damn Vulnerable Web Application (DVWA)
Target	http://127.0.0.1:42001/
Security Level	Low
Backend	MySQL / MariaDB
Tools	Firefox, SQLMap
Environment	Local controlled laboratory

3. Methodology

- Configured DVWA and created/reset the local database.
- Set DVWA security level to Low.
- Performed manual SQL injection testing.
- Used SQLMap for injection detection and database/table/data enumeration.
- Tested authentication bypass against the DVWA login form.
- Tested GET parameter tampering.
- Tested boolean-based blind SQL injection.
- Reviewed impact and prepared remediation recommendations.

4. Findings Summary

Finding	Result	Evidence	Impact
SQL Injection — GET id	Confirmed	Manual + SQLMap	Database records can be exposed/manipulated.
SQLMap enumeration	Confirmed	Evidence 2–3	Database structure disclosed.
Users table exposure	Confirmed	Evidence 3	Credential-related data retrievable.
GET parameter tampering	Confirmed	Evidence 4	Different user record retrieved.

Boolean-based Blind SQLi	Confirmed	Evidence 5–7	True/false conditions distinguishable.
Time-based Blind SQLi	Confirmed	SQLMap output	Time-delay inference detected.
Authentication bypass	Not reproduced	Manual test	Classic SQLi login bypass unsuccessful.

5. Detailed Results

5.1 Manual SQL Injection

A boolean SQL injection payload was supplied to the SQL Injection module. The application returned multiple user records instead of a single record, confirming SQL injection in the controlled DVWA target.

Evidence 1 — Manual SQL Injection

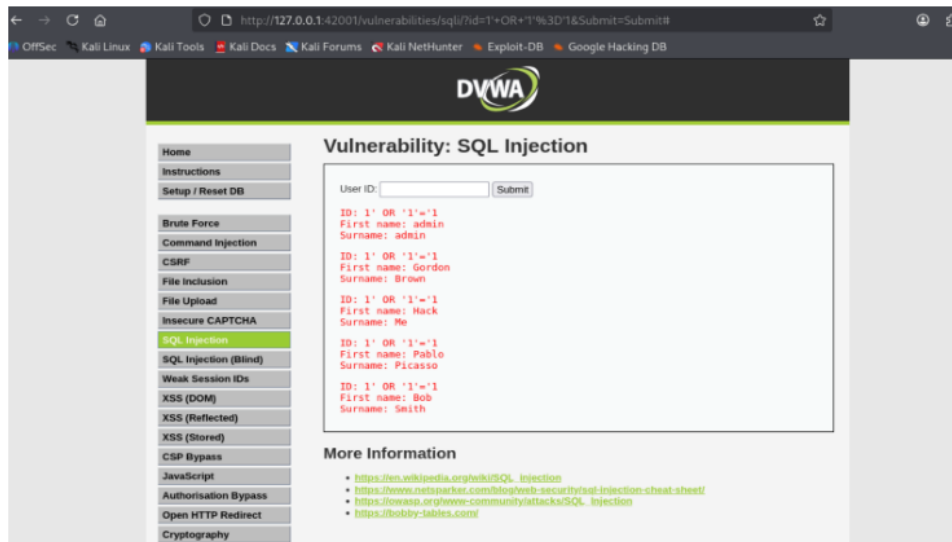
			exposed/manipulated.
SQLMap enumeration	Confirmed	Evidence 2–3	Database structure disclosed.
Users table exposure	Confirmed	Evidence 3	Credential-related data retrievable.
GET parameter tampering	Confirmed	Evidence 4	Different user record retrieved.
Boolean-based Blind SQLi	Confirmed	Evidence 5–7	True/false conditions distinguishable.
Time-based Blind SQLi	Confirmed	SQLMap output	Time-delay inference detected.
Authentication bypass	Not reproduced	Manual test	Classic SQLi login bypass unsuccessful.

5. Detailed Results

5.1 Manual SQL Injection

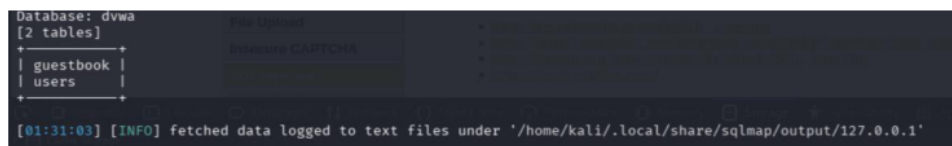
A boolean SQL injection payload was supplied to the SQL Injection module. The application returned multiple user records instead of a single record, confirming SQL injection in the controlled DVWA target.

Evidence 1 — Manual SQL Injection



5.2 SQLMap — Database and Table Enumeration

SQLMap identified the GET parameter 'id' as injectable and identified MySQL/MariaDB as the backend. The dvwa database contained the tables 'guestbook' and 'users'.



Module 5 — Web Application Security Assessment and SQL Injection Testing

5.2 SQLMap — Database and Table Enumeration

SQLMap identified the GET parameter 'id' as injectable and identified MySQL/MariaDB as the backend. The dvwa database contained the tables 'guestbook' and 'users'.

5.3 SQLMap — Users Data Extraction

Module 5 — Web Application Security Assessment and SQL Injection Testing

The users table was selected and successfully dumped. The result demonstrated exposure of user and credential-related information. Sensitive credential material has been redacted in the evidence below.

5.3 SQLMap — Users Data Extraction

The users table was selected and successfully dumped. The result demonstrated exposure of user and credential-related information. Sensitive credential material has been redacted in the evidence below.

```
01:34:29 [INFO] (root) testing database-based cracking (old, generic, unsafe)
[INFO] (root) request output

Database: dvwa
Table: users
5 entries

+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | user | avatar | password | last_name | first_name | last_login | failed_log |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | /hackable/users/admin.jpg | [REDACTED] | admin | admin | 2026-10-02 00:54:54 | 0 |
| 2 | gordonb | /hackable/users/gordonb.jpg | [REDACTED] | Brown | Gordon | 2026-10-02 00:54:54 | 0 |
| 3 | 1337 | /hackable/users/1337.jpg | [REDACTED] | Me | Hack | 2026-10-02 00:54:54 | 0 |
| 4 | pablo | /hackable/users/pablo.jpg | [REDACTED] | Picasso | Pablo | 2026-10-02 00:54:54 | 0 |
| 5 | smithy | /hackable/users/smithy.jpg | [REDACTED] | Smith | Bob | 2026-10-02 00:54:54 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+

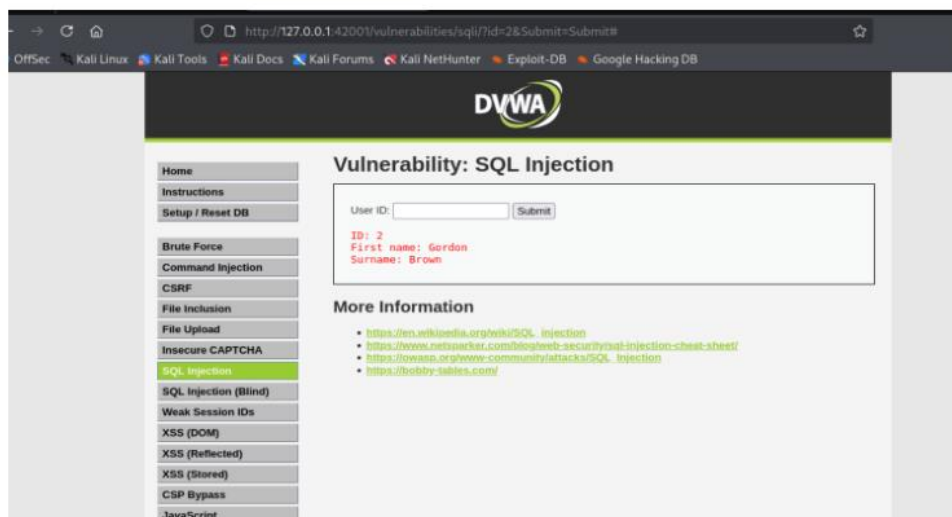
01:34:29 [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/127.0.0.1/dump/dvwa/users.csv'
01:34:29 [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/127.0.0.1'
*) ending @ 01:34:29 /2026-10-02/
```

5.4 Authentication Bypass

A classic SQL injection authentication-bypass attempt was tested against the DVWA login form. The attempt did not result in a successful login. Therefore, authentication bypass is recorded as tested but not reproduced.

5.5 GET Parameter Tampering

Changing the GET parameter id from one identifier to another returned a different user record. For example, id=2 returned Gordon Brown. This demonstrates that the application directly trusted a user-controlled identifier.



5.4 Authentication Bypass

A classic SQL injection authentication-bypass attempt was tested against the DVWA login form. The attempt did not result in a successful login. Therefore, authentication bypass is recorded as tested but not reproduced.

5.5 GET Parameter Tampering

Changing the GET parameter id from one identifier to another returned a different user record. For example, id=2 returned Gordon Brown. This demonstrates that the application directly trusted a user-controlled identifier.

5.6 Boolean-Based Blind SQL Injection

The blind SQL injection module returned different responses for logically true and false conditions. The true condition returned 'User ID exists in the database', while the false condition returned 'User ID is MISSING from the database'.

Evidence 5 — Baseline

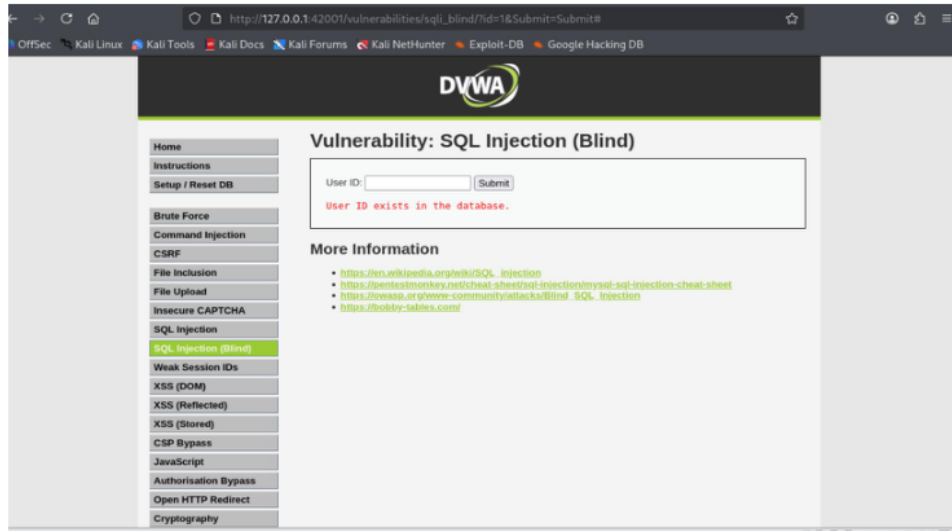
Evidence 6 — True condition

Evidence 7 — False condition

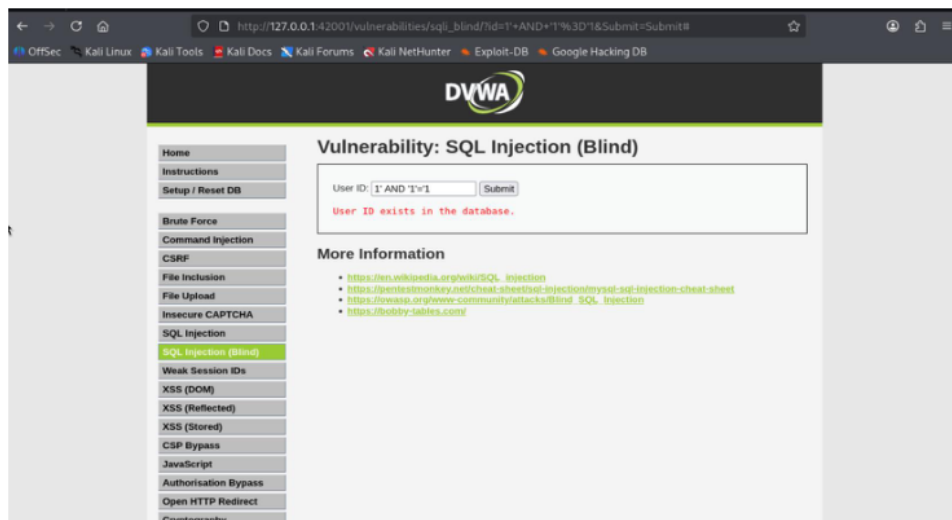
5.6 Boolean-Based Blind SQL Injection

The blind SQL injection module returned different responses for logically true and false conditions. The true condition returned 'User ID exists in the database', while the false condition returned 'User ID is MISSING from the database'.

Evidence 5 — Baseline



Evidence 6 — True condition



Evidence 7 — False condition

Module 5 — Web Application Security Assessment and SQL Injection Testing

5.7 Time-Based Blind SQL Injection

SQLMap identified the GET parameter id as vulnerable to MySQL time-based blind SQL injection using a SLEEP-based technique. This confirms that database conditions could be inferred through response timing in the controlled lab.

Evidence 8 — SQLMap Time-Based Blind SQL Injection

Module 5 — Web Application Security Assessment and SQL Injection Testing

```
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 9559 FROM (SELECT(SLEEP(2)))ACnc) AND 'zFsb'='zFsb&Submit=Submit
[03:02:48] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.30.1
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
```

6. Impact

- Disclosure of database structure and application data.
- Unauthorized retrieval of users-table records.
- Exposure of credential-related data.
- Potential database manipulation depending on query context and privileges.
- Boolean-based inference of database conditions.
- Time-based inference through delayed responses.
- Access to another user's record by changing a user-controlled identifier.

7. Remediation Recommendations

- Use parameterized queries / prepared statements for all database operations.
- Never concatenate user-controlled input directly into SQL statements.
- Apply strict server-side input validation and allow-list expected formats.
- Use least-privilege database accounts.
- Store passwords using modern password-hashing algorithms such as Argon2id or bcrypt.
- Do not expose database errors or unnecessary internal details to users.
- Perform authorization checks before returning records selected by user-controlled IDs.
- Use secure session and authentication controls.
- Log and monitor suspicious SQL injection patterns.
- Retest after remediation to confirm that injection is no longer possible.

8. Conclusion

The controlled DVWA assessment successfully demonstrated SQL injection, SQLMap enumeration and data extraction, GET parameter tampering, boolean-based blind SQL injection, and time-based blind SQL injection. The users table was successfully extracted, demonstrating significant data exposure. Authentication bypass was tested but not reproduced, and the report does not claim a successful bypass. All testing was performed against the intentionally vulnerable local DVWA environment.

9. Evidence Index

Evidence	Description
1_SQLInjection_Manual.png	Manual SQL injection returning multiple records
2_SQLMap_Tables.png	SQLMap enumeration of DVWA tables
3_SQLMap_Data_Dump_REDACTED.png	Users table extraction with credential material redacted
4_GET_Parameter_Tampering.png	GET id parameter tampering
5_Blind_SQLInjection_Baseline.png	Blind SQLi baseline
6_Blind_SQLInjection_True.png	Boolean true condition
7_Blind_SQLInjection_False.png	Boolean false condition
8_SQLMap_TimeBased_Blind.png	SQLMap confirmation of MySQL time-based blind SQL injection using SLEEP